

Hard Landing Detection System

What is Hard Landing Detection?

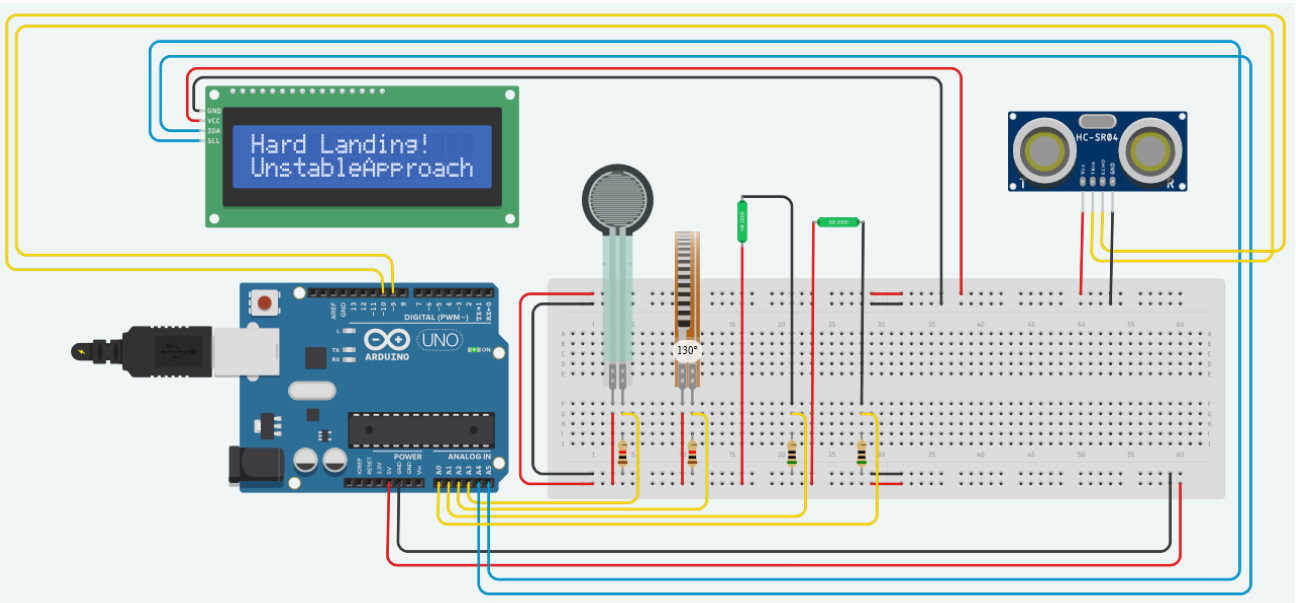
Hard Landing Detection (HLD) is a system designed to detect when a device, such as a drone, robot, or package, has experienced *a severe impact or drop*, potentially causing damage.

In the *civil aviation industry*, hard landing is a typical unsafety event of great concern to airlines and civil aviation authorities, as their misidentification can potentially lead to catastrophic accidents.

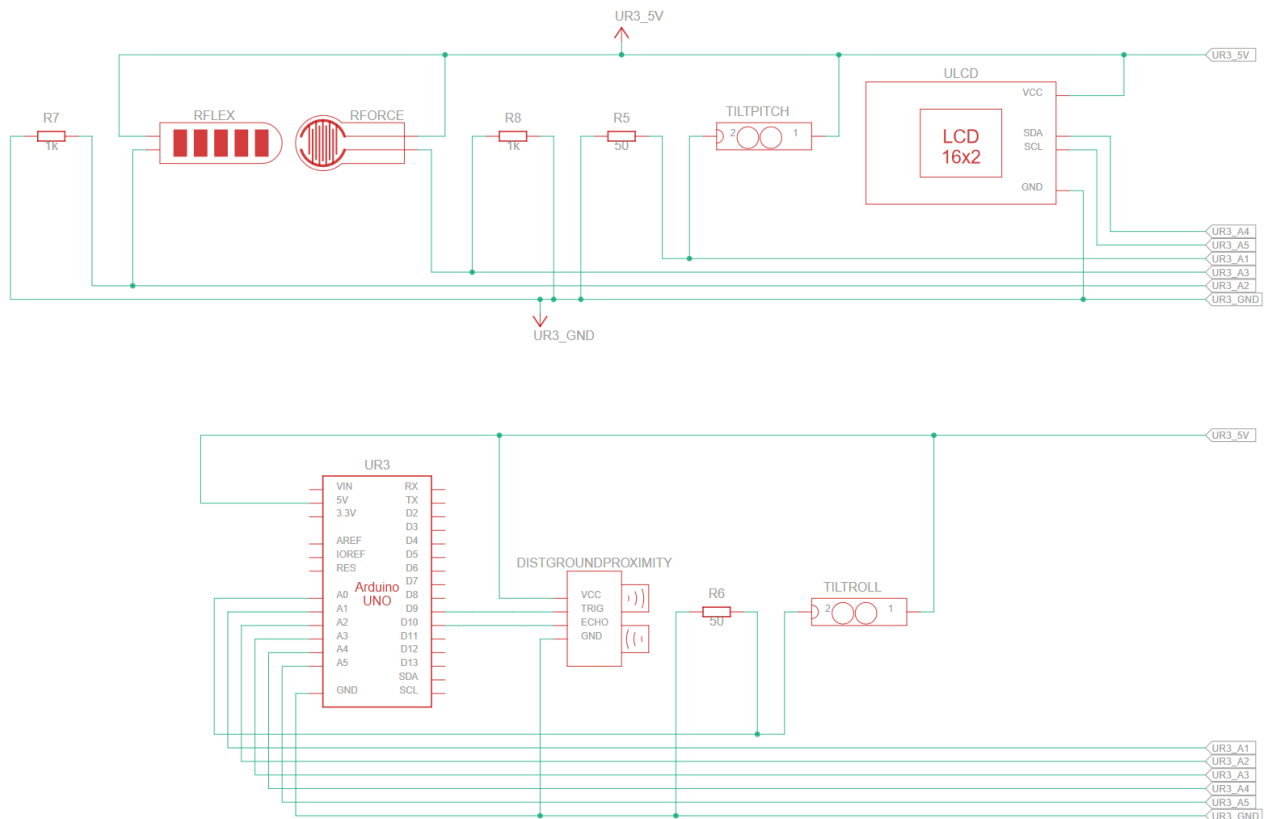
Components Required

Name	Quantity	Component
TILTPitch TILTRoll	2	Tilt Sensor
RForce	1	Force Sensor
RFlex	1	Flex Sensor
UR3	1	Arduino Uno R3
R8 R7	2	1 kΩ Resistor
ULCD	1	MCP23008-based, 32 (0x20) LCD 16 x 2 (I2C)
R5 R6	2	50 Ω Resistor
DISTGroundProximity	1	Ultrasonic Distance Sensor (4-pin)

Connections



Schematics



Detection Method

1. **Threshold-based detection:** Set threshold values for each sensor (e.g., altitude change, flexure, force, tilt angle).
2. **Sensor fusion:** Combine data from multiple sensors to determine the severity of the impact.

Advantages

1. **Prevents damage:** Detects potential damage, allowing for timely maintenance or repair.
2. **Reduces downtime:** Minimizes device downtime and increases overall efficiency.
3. **Improves safety:** Identifies potential safety risks, ensuring safe operation.
4. **Optimizes maintenance:** Enables predictive maintenance, reducing maintenance costs.

How does it help maintenance?

1. **Predictive maintenance:** Schedules maintenance based on detected impacts.
2. **Damage assessment:** Provides insights into the severity of damage.
3. **Reduced inspection time:** Focuses maintenance efforts on affected areas.

Use cases

1. **Drone delivery:** Detects hard landings during package delivery.

1. **Aviation Safety:** Detecting hard landings for maintenance alerts, reducing wear and tear on landing gear.
2. **Aircraft Health Monitoring:** Providing real-time impact data to assess structural integrity and prevent fatigue damage.
3. **Pilot Training and Feedback:** Offering instant feedback on landing quality, helping pilots improve landing techniques.
4. **Predictive Maintenance:** Enabling data-driven scheduling of repairs and inspections based on hard landing frequency.
5. **Autonomous Flight Systems:** Assisting autonomous aircrafts to achieve softer landings by monitoring impact.

CODE

```
#include <Wire.h>
#include <Adafruit_LiquidCrystal.h>

bool hardLanding = false;
bool unstableApproach = false;

const int forceSensor = A3;           // Pin A3
const int flexSensor = A2;           // Pin A2
const int tiltSensorPitch = A1;      // Pin A1
const int tiltSensorRoll = A0;       // Pin A0
const int trigPin = 9;               // Trigger pin : pin 9
const int echoPin = 10;              // Echo pin : pin 10

// store the duration and distance
long duration = 0;
float distance = 0;
float force = 0;
float flex = 0;
float pitch = 0;
float roll = 0;
// store the threshold values
int thresholdForce = 245;
int thresholdFlex = 27;
int thresholdPitch = 852;
int thresholdRoll = 852;

Adafruit_LiquidCrystal lcd(0);

int getDistance() {
  // Clear the trigPin by setting it LOW for 2 microseconds
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  // Trigger the sensor by setting the trigPin HIGH for 10 microseconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
```

```

    // Read the echoPin, returns the time in microseconds the pulse took to
    return
    duration = pulseIn(echoPin, HIGH);
    // Calculate the distance
    return duration * 0.0343 / 2;
}

int readForceSensor() {
    return analogRead(forceSensor);
}

// Function to read the flex sensor value
int readFlexSensor() {
    return analogRead(flexSensor);
}

// Function to read the tilt sensor pitch value
int readTiltSensorPitch() {
    return analogRead(tiltSensorPitch);
}

// Function to read the tilt sensor roll value
int readTiltSensorRoll() {
    return analogRead(tiltSensorRoll);
}

void setup() {
    Serial.begin(9600);
    lcd.begin(16, 2);
    lcd.setBacklight(1);
    // Set trigPin as output and echoPin as input
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
}

void loop() {
    distance = getDistance();

    if (distance <= 100) {
        int force = readForceSensor();
        int flex = readFlexSensor();
        int pitch = readTiltSensorPitch();
        int roll = readTiltSensorRoll();

        // Check if hardLanding hasn't been detected yet
        if (!hardLanding && flex <= thresholdFlex && force >= thresholdForce) {
            Serial.println("Hard Landing!");
            lcd.setCursor(0, 0);           // Set cursor to column 0, row 0
            lcd.print("Hard Landing!");
            hardLanding = true;           // Set the flag to true to stop
re-checking
        }
    }
}

```

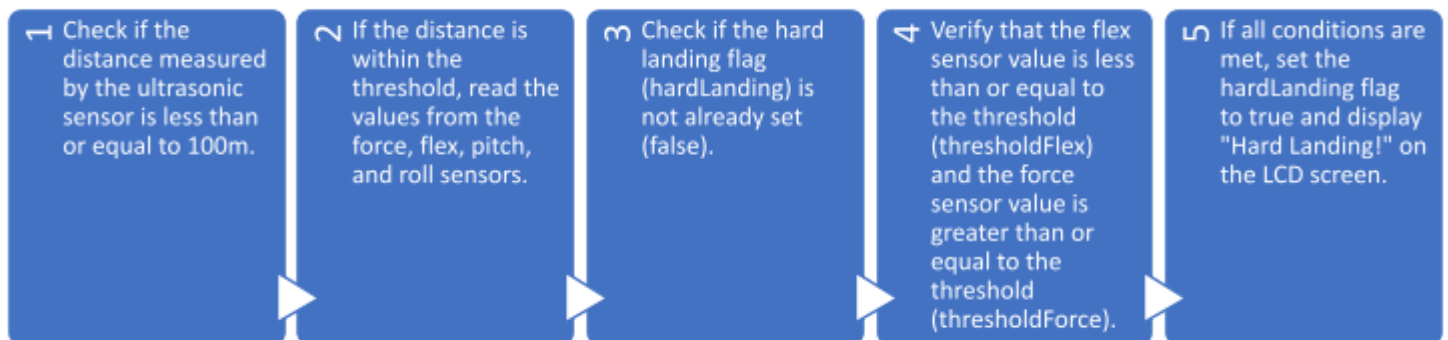
```

    // Check if unstableApproach hasn't been detected yet
    if (!unstableApproach && (pitch >= thresholdPitch || roll >=
thresholdRoll)) {
        Serial.println("UnstableApproach");
        lcd.setCursor(0, 1);           // Set cursor to column 0, row 1
        lcd.print("UnstableApproach");
        unstableApproach = true;       // Set the flag to true to stop
re-checking
    }

    delay(500);
}
}

```

The logic used to detect hard landing in the provided code is as follows:



Threshold Values:

- thresholdForce: 245
- thresholdFlex: 27

Sensor Values:

- force: value from force sensor (A3)
- flex: value from flex sensor (A2)

Decision Table:

Distance	Hard Landing Flag	Flex Sensor	Force Sensor	Decision
≤ 100	false	≤ thresholdFlex	≥ thresholdForce	Hard Landing Detected
≤ 100	true	-----	-----	No Action
> 100	false	-----	-----	No Action

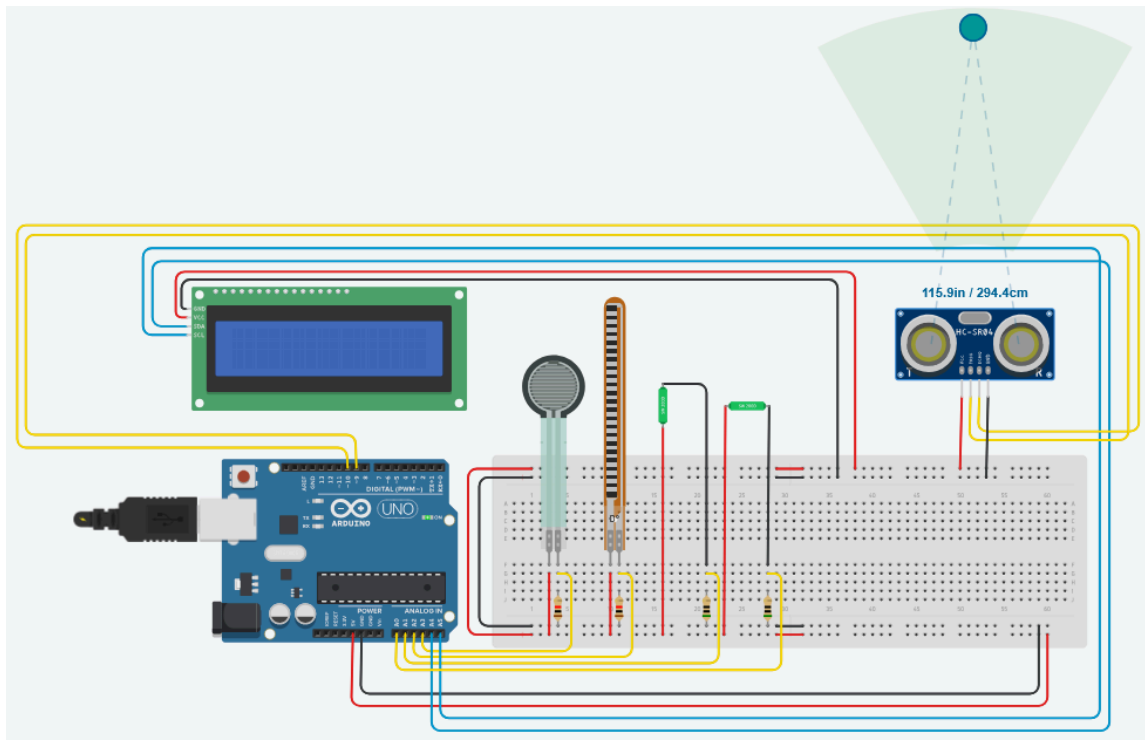
This logic detects hard landings based on the combination of:

1. **Proximity** (distance ≤ 100)
2. **Low flex sensor value** (indicating compression)
3. **High force sensor value** (indicating impact)

By using these conditions, the code effectively detects hard landings and prevents repeated detections by setting the hardLanding flag.

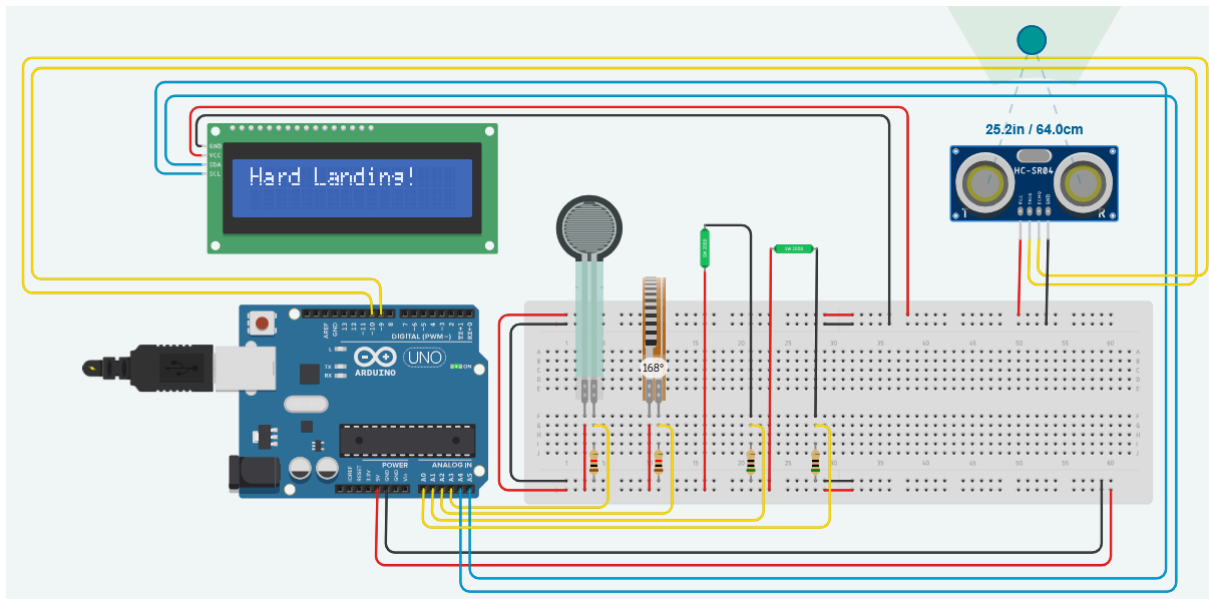
Case 1:

In Air: detection system off

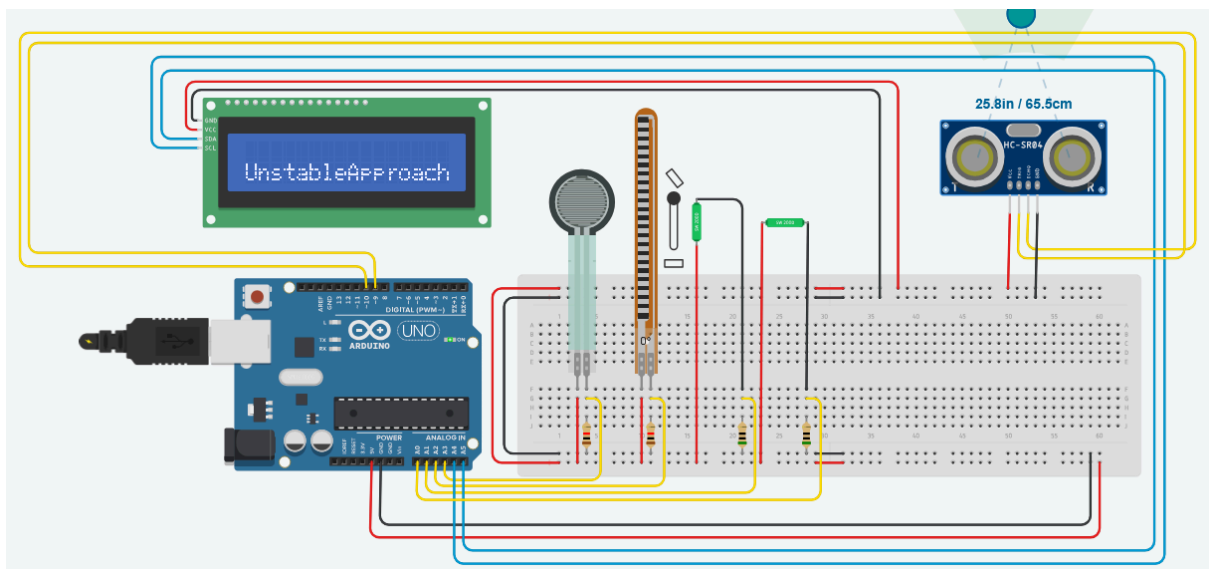


Case 2:

On ground mode: excess force and flex detected



Case 3:
Landing Mode: Unstable Approach



Case 4:
Combination of Case 2 and Case 3

